

第4章 贪心法

提纲

CONTENTS

4.1 贪心法概述

4.2 求解组合问题

4.3 求解图问题

4.4 求解调度问题

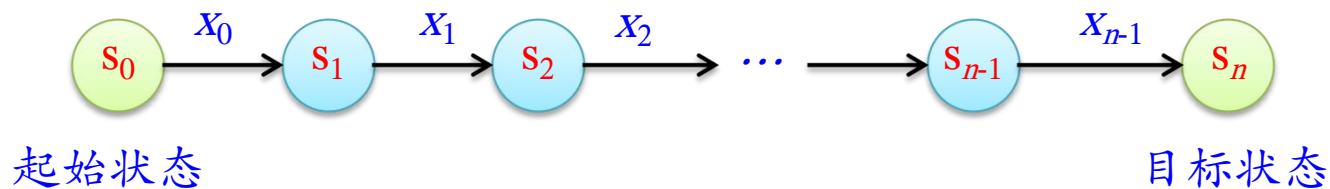
4.5 哈夫曼编码

4.6 轮船装载问题

4.1 贪心法概述

4.1.1 什么是贪心法

■ 解向量 $x = (x_0, x_1, \dots, x_{n-1})$



■ 每一步总是做出在当前看来是最好的选择，也就是说贪心法不从整体最优上考虑，所做出的仅是在某种意义上的局部最优解。



- 每一步的局部最优选择仅依赖以前的决策，且不依赖于以后的决策。
- 所有局部最优解合起来不一定构成整体最优解。所以贪心法不能保证对所有问题都得到整体最优解。
- 因此采用贪心法求最优解时，必须证明该算法的每一步上做出的选择都必然得到整体最优解。

4.1.2 贪心法求解问题具有的性质

1. 最优子结构性质

- 如果一个问题的最优解包含其子问题的最优解，则称此问题具有**最优子结构性质**。
- 证明方法：采用**反证法**，先假设由问题的最优解导出的子问题的解不是最优的，然后证明在这个假设下可以构造出比原问题的最优解更好的解，从而导致矛盾。

2. 贪心选择性质

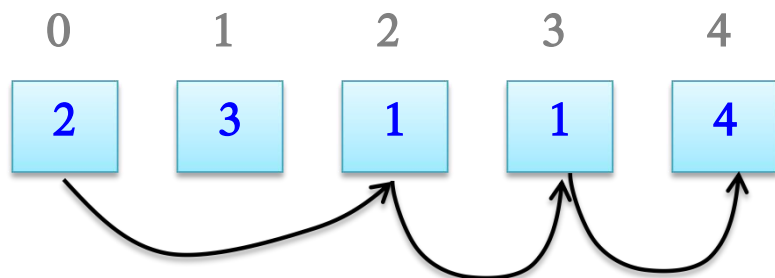
- 指整体最优解可以通过一系列局部最优选择（即贪心选择）来得到。
- 证明方法：采用数学归纳法证明，即证明每一步所做的贪心选择最终得到问题的整体最优解。

【例4-1】 (LeetCode45) 给定一个含 n ($1 \leq n \leq 1000$) 个非负整数数组 `nums` ($0 \leq \text{nums}[i] \leq 1000$)，数组中的每个元素表示在该位置可以跳跃的最大长度，假设总是可以从初始位置0到达最后一个位置 $n-1$ ，设计一个算法求最少的跳跃次数。

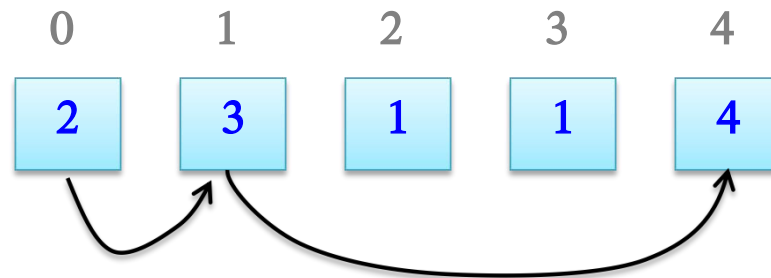
例如`nums={2, 3, 1, 1, 4}`， $n=5$ ，从位置0可以跳1步到达位置1，再从位置1跳3步到达位置4，所以结果为2。

解

- **nums[i]**含义：如果从位置*i*能够跳到位置*j* ($j \geq i$)，则从位置*i*一定可以跳到*[i, j-1]*的任何位置。
- 采用**简单贪心选择策略**即每次从位置*i*跳最大步数**nums[i]**到达**nums[i]+i**，这样能不能得到最少的跳跃次数，答案是否定的。
- 例如**nums = (2, 3, 1, 1, 4)**，**n=5**

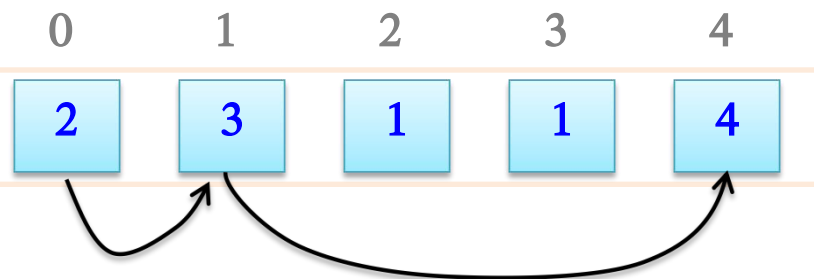


- 从位置0跳1次到达位置2，
- 从位置2跳1次只能达位置3，
- 从位置3跳1次只能达位置4，
- 结果是跳跃3次。

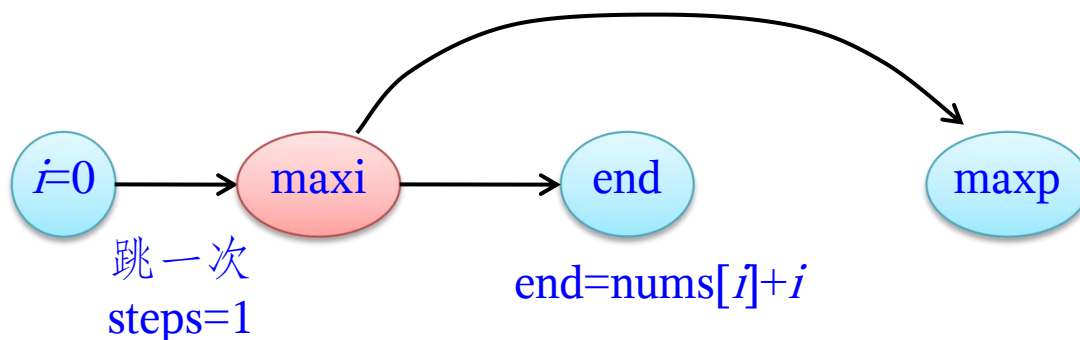


- 从位置0跳1次到达位置1（跳1步）
- 从位置1跳1次到达位置4（跳3步）
- 只需要跳跃2次。

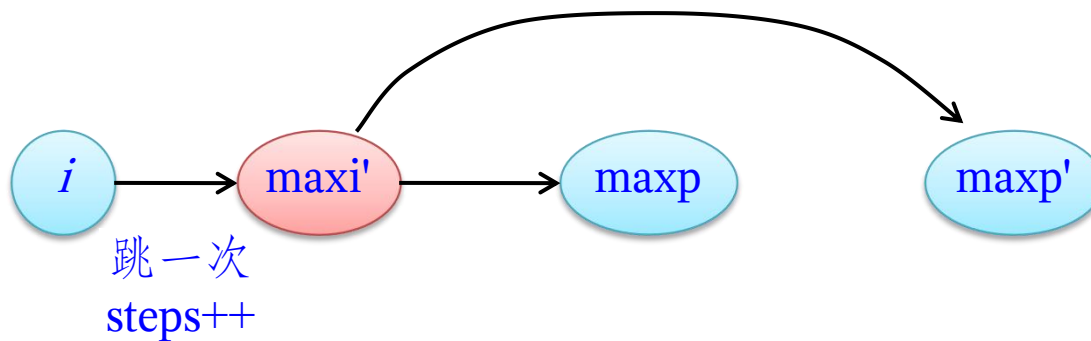
■ 正确的贪心选择策略



$[i, \text{end}]$ 中跳1次能够到达的最远位置 maxp



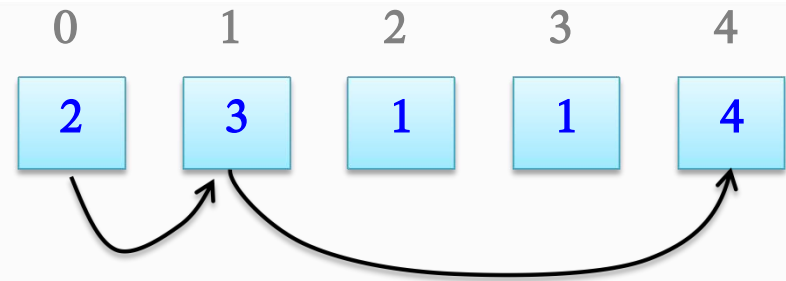
↓ 子问题 $i = \text{maxi}$




```

int jump(vector<int>& nums)
{
    int n=nums.size(); if(n==1) return 0;
    int end=nums[0]; int maxp; int steps=1;
    printf("从位置0跳一次,steps=%d\n",steps);
    int i=1,maxi;
    while(i<n-1)
    {
        maxp=0;
        while(i<n-1)
        {
            if(maxp<nums[i]+i)
            {
                maxp=nums[i]+i;
                maxi=i;
            }
            if(i==end) break;
            i++;
        }
        if(i<n-1)
        {
            i=maxi; end=maxp; steps++;
            printf("从位置%d跳一次,steps=%d\n",maxi,steps);
        }
    }
    return steps;
}

```



//遍历nums[0..n-2]

//求[i..end]能够跳到的最远位置maxp

//maxi为最远位置maxp的起跳位置

//[i,end]处理完毕退出循环

//位置i有效则跳一次跳到maxp位置

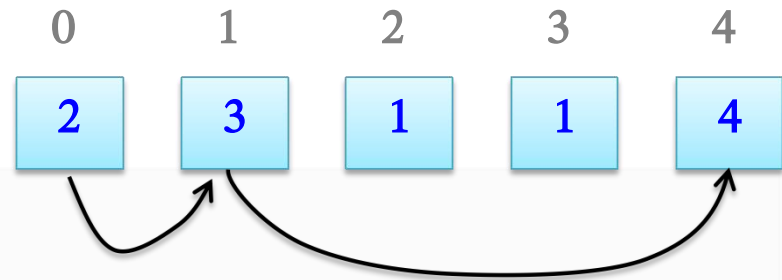
执行结果：通过 显示详情 >

执行用时：12 ms，在所有 C++ 提交中击败了 80.31% 的用户

内存消耗：16 MB，在所有 C++ 提交中击败了 39.91% 的用户

通过测试用例：106 / 106

等效的算法（仅求最少跳跃次数）



```
int jump(vector<int>& nums)
```

```
{  int n=nums.size();
```

```
    int end=0;
```

```
    int maxp=0;
```

```
    int steps=0;
```

```
    for(int i=0;i<n-1;i++)
```

```
    {  maxp=max(maxp,nums[i]+i);
```

```
        if(i==end)
```

```
        {  end=maxp;
```

```
            steps++;
```

```
        }
```

```
    }
```

```
    return steps;
```

```
}
```

//表示[i..end]区间的末端

//[i..end]中任意位置起跳最多到达的位置

//最少跳跃次数

//取位置i的最大跳跃长度

//区间[i..end]处理完

//新区间为[i+1..maxp]

//跳跃次数增加1

执行结果： **通过** [显示详情](#)

执行用时： 12 ms，在所有 C++ 提交中击败了 80.31% 的用户

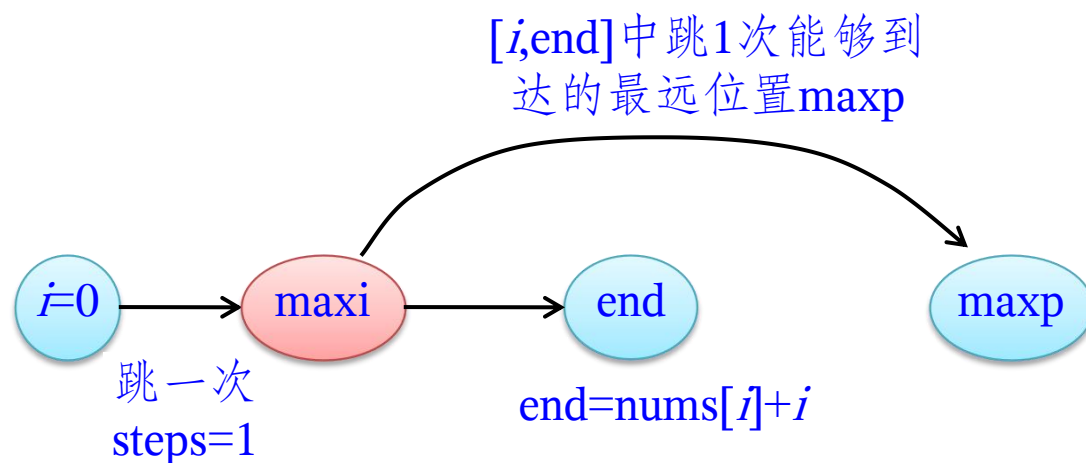
内存消耗： 15.9 MB，在所有 C++ 提交中击败了 76.49% 的用户

通过测试用例： 106 / 106

证明上述贪心算法得到的steps一定是最少跳跃次数。

① 最优子结构性质。

反证法证明，假设最优解是 $steps = steps1 + 1$ ，第一次跳到 $maxi$ ，而 $steps1$ 不是子序列 $nums[maxi..n-1]$ 的最优解，说明该子序列有一个更优解 $step2 < step1$ ，这样 $steps = step2 + 1$ 更小，与 $steps$ 是 $nums$ 的最优解矛盾。

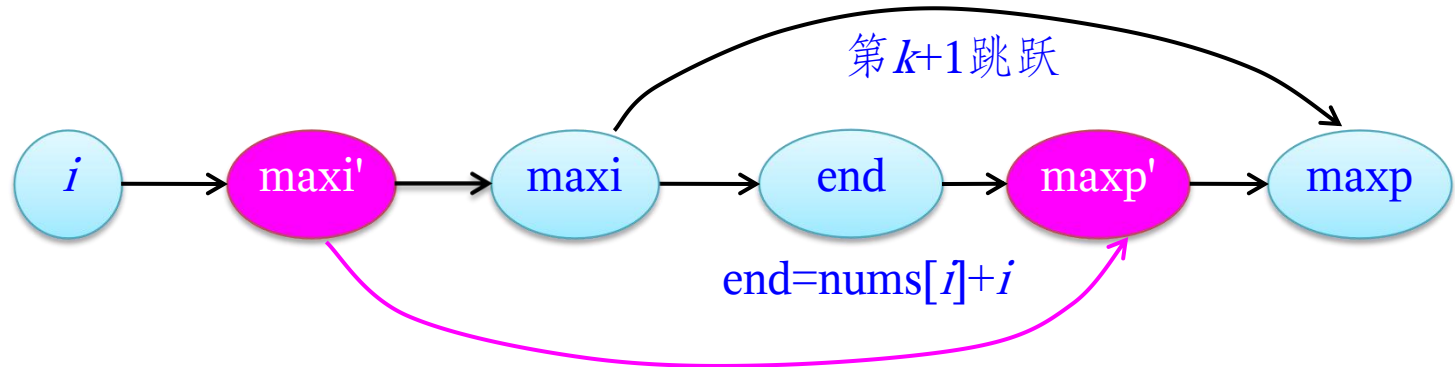


② 贪心选择性质。采用数学归纳法证明。

第一次跳跃按最优子结构性质的证明过程看出是最优的。

假设第 k 次跳跃是最优的，其起点位置为 i ， $\text{end}=\text{nums}[i]+i$ 。在 $[i, \text{end}]$ 中找到位置 maxi ，该位置可以跳到最远位置 maxp ，则第 $k+1$ 次跳跃应该是从 maxi 位置开始跳跃一次。

假设第 $k+1$ 次的上述跳跃不是最优的，即不是从 maxi 而是从 maxi' 开始， $\text{maxp}'=\text{nums}[\text{maxi}']+\text{maxi}'$ ，显然 $\text{maxp}'<\text{maxp}$ ，这样从 maxi' 跳跃到终点的跳跃次数一定不少于从 maxi 跳跃到终点的跳跃次数，与最优跳跃的假设矛盾。



4.1.3 贪心法的一般求解过程

- ① 建立数学模型来描述问题。
- ② 把求解的问题分成若干个子问题。
- ③ 对每一子问题求解，得到子问题的局部最优解。
- ④ 把子问题的解局部最优解合成原问题的一个最优解。

贪心法求解问题的算法框架

```
SolutionType Greedy(SType a[],int n)
```

```
{   SolutionType x={};
```

```
    for (int i=0;i<n;i++)
```

```
    {   SType xi=Select(a);
```

```
        if (Feasible(xi))
```

```
            x=Union(xi);
```

```
    }
```

```
    return x;
```

```
}
```

```
//初始时，解向量为空
```

```
//执行n步操作
```

```
//从输入a中选择一个当前最好的分量
```

```
//判断xi是否包含在当前解中
```

```
//将xi分量合并形成x
```

```
//返回生成的最优解
```

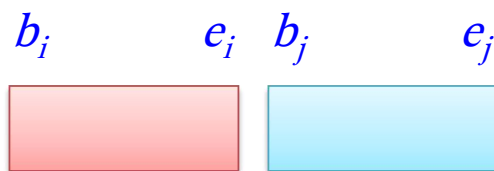
4.2 求解组合问题

4.2.1 活动安排问题 I

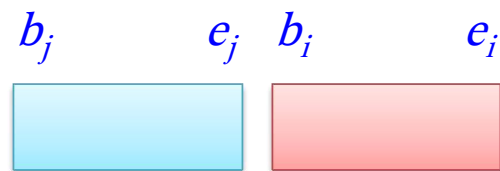
- 假设有 n 个活动 $S = (1, 2, \dots, n)$ ，有一个资源，每个活动执行时都要占用该资源，并且该资源任何时刻只能被一个活动所占用，一旦某个活动开始执行，中间不能被打断，直到其执行完毕。每个活动 i 有一个开始时间 b_i 和结束时间 e_i ($b_i < e_i$)，它是一个半开时间区间 $[b_i, e_i)$ ，假设最早活动执行时间为 0。
- 求一种最优活动安排方案，使得安排的活动个数最多。

解

- 对于两个活动 i 和 j , 若满足 $b_j \geq e_i$ 或 $b_i \geq e_j$, 则它们是不重叠的, 称为两个兼容活动。



(a) $b_j \geq e_i$



(b) $b_i \geq e_j$

- 本问题就是在 n 个活动中选择最多的兼容活动即求最多兼容活动的个数。

- 用数组A存放所有的活动， $A[i].b$ ($1 \leq i \leq n$) 存放活动起始时间， $A[i].e$ 存放活动结束时间。
- **贪心策略**：每一步总是选择执行这样的一个活动，它能够使得余下的活动的时间最大化即余下活动中兼容活动尽可能多。
- 为此先**按活动结束时间递增排序**，再从开头开始依次选择兼容活动（用B集合表示），从而得到最大兼容活动子集即包含兼容活动个数最多的子集。

i	1	2	3	4	5	6	7	8	9	10	11
开始时间b	1	3	0	5	3	5	6	8	8	2	12
结束时间e	4	5	6	7	8	9	10	11	12	13	15

求最大兼容活动集B的过程

- $i=1$: $preend=0$, 活动1[1, 4)的开始时间大于0, 选择它, $preend=$ 活动1的结束时间=4, $B=\{1\}$ 。
- $i=2$: 活动2[3, 5)的开始时间小于 $preend$, 不选取。
- $i=3$: 活动3[0, 6)的开始时间小于 $preend$, 不选取。
- $i=4$: 活动4[5, 7)的开始时间大于 $preend$, 选择它, $preend=7$, $B=\{1, 4\}$ 。
- $i=5$: 活动5[3, 8)的开始时间小于 $preend$, 不选取。

i	1	2	3	4	5	6	7	8	9	10	11
开始时间b	1	3	0	5	3	5	6	8	8	2	12
结束时间e	4	5	6	7	8	9	10	11	12	13	15

- $i=6$: 活动6[5, 9)的开始时间小于preend, 不选取。
- $i=7$: 活动7[6, 10)的开始时间小于preend, 不选取。
- $i=8$: 活动8[8, 11)的开始时间大于preend, 选择它, preend=11, B={1, 4, 8}。
- $i=9$: 活动9[8, 12)的开始时间小于preend, 不选取。
- $i=10$: 活动10[2, 13)的开始时间小于preend, 不选取。
- $i=11$: 活动11[12, 14)的开始时间大于preend, 选择它, preend=14, B={1, 4, 8, 11}。

```
struct Action
{
    int b;
    int e;
    bool operator<(const Action &s) const
    {
        return e<=s.e;
    }
};
```

//活动类型

//活动起始时间

//活动结束时间

//用于按活动结束时间递增排序

```
bool flag[MAX];
```

//标记选择的活动

```

void greedily(vector<Action>& A)    //求解最大兼容活动子集
{
    int n=A.size();
    memset(flag,0,sizeof(flag));    //初始化为false

    sort(A.begin(),A.end());        //A按活动结束时间递增排序

    int preend=0;                    //前一个兼容活动的结束时间
    for (int i=0;i<n;i++)
    {
        if (A[i].b>=preend)
        {
            flag[i]=true;            //选择A[i]活动
            preend=A[i].e;
        }
    }
}

```

【算法分析】 算法的主要时间花费在排序上，排序时间为 $O(n\log_2 n)$ ，所以整个算法的时间复杂度为 $O(n\log_2 n)$ 。

最优子结构性质

- 所有活动按结束时间递增排序，这里就是要证明若 X 是 A 的最优解， $X = X' \cup \{1\}$ ，则 X' 是 $A' = \{i \in A: e_i \geq b_1\}$ 的最优解。
- 那么 A 是不是总存在一个以活动1开始的最优解。如果第一个选择的活动的活动为 k ($k \neq 1$)，可以构造另一个最优解 Y ， Y 与 X 的活动数相同。那么在 Y 中用活动1取代活动 k 得到 Y' ，因为 $e_1 \leq e_k$ ，所以 Y' 中活动也是兼容的，即 Y' 也是最优解，这就说明 A 总存在一个以活动1开始的最优解。
- 当选择活动1后，原问题就变成了在 A' 中找兼容活动的子问题。如果 X 为原问题的一个最优解，而 $X' = X - \{1\}$ 不是 A' 的一个最优解，说明 A' 能够找到的一个更优解 Y' ， Y' 中的兼容活动个数多于 X' ，这样将活动1加入 Y' 后就得到 A 的一个更有解 Y ， Y 中兼容活动个数多于 X ，这就与 X 是最优解的假设相矛盾。

贪心选择性质

从前面最优子结构性证明可以看出，每一步所做的贪心选择都将问题简化为一个更小的与原问题具有相同形式的子问题，可以对贪心选择次数用数学归纳法证明，这里不再详述。

4.2.2 求解背包问题

- 有 n 个编号为 $0 \sim n-1$ 的物品，重量为 $w=\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $v=\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。
- 从这些物品中选取全部或者部分物品装入该背包中，找到选中物品不仅能够放到背包中而且价值最大的方案。
- 与0/1背包问题的区别是这里的每个物品可以取一部分装入背包。

物品编号no	0	1	2	3	4
w	10	20	30	40	50
v	20	30	66	40	60

$W=100$

解

■ 贪心策略：每次选择单位重量价值最大的物品。

序号 i	0	1	2	3	4
物品编号 no	3	1	2	5	4
w	30	10	20	50	40
v	66	20	30	60	40
v/w	2.2	2.0	1.5	1.2	1.0

$x[0]=1$
 $bestv=66$
 $rw=rw-w[0]=70$

$x[1]=1$
 $bestv=66+20=86$
 $rw=rw-w[1]=60$

$x[2]=1$
 $bestv=86+30=116$
 $rw=rw-w[2]=40$

$x[3]=0.8$
 $bestv=116+0.8*60=164$

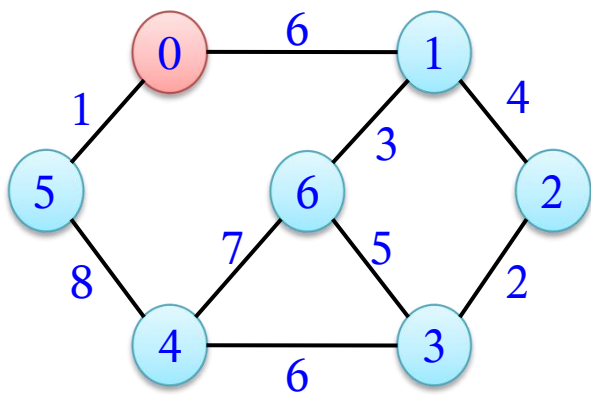
<code>void knap()</code>	<code>//贪心法求解背包问题</code>
<code>{ bestv=0;</code>	<code>//bestv初始化为0</code>
<code>double rw=W;</code>	<code>//背包中能装入的余下重量</code>
<code>memset(x,0,sizeof(x));</code>	<code>//初始化x向量</code>
<code>int i=0;</code>	
<code>while (i<n && A[i].w<rw)</code>	<code>//物品i能够全部装入时循环</code>
<code>{ x[i]=1;</code>	<code>//装入物品i</code>
<code>rw-=A[i].w;</code>	<code>//减少背包中能装入的余下重量</code>
<code>bestv+=A[i].v;</code>	<code>//累计总价值</code>
<code>i++;</code>	<code>//继续循环</code>
<code>}</code>	
<code>if (i<n && rw>0)</code>	<code>//当余下重量大于0</code>
<code>{ x[i]=rw/A[i].w;</code>	<code>//将物品i的一部分装入</code>
<code>bestv+=x[i]*A[i].v;</code>	<code>//累计总价值</code>
<code>}</code>	
<code>}</code>	

【算法分析】 排序算法`sort()`的时间复杂性为 $O(n\log_2 n)$ ，`while`循环的时间为 $O(n)$ ，所以算法的时间复杂度为 $O(n\log_2 n)$ 。

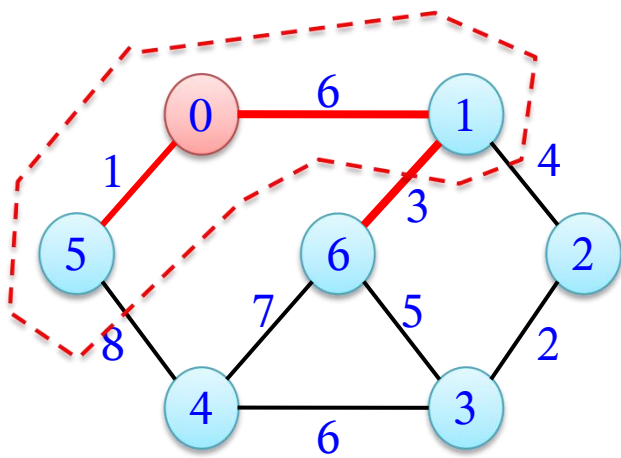
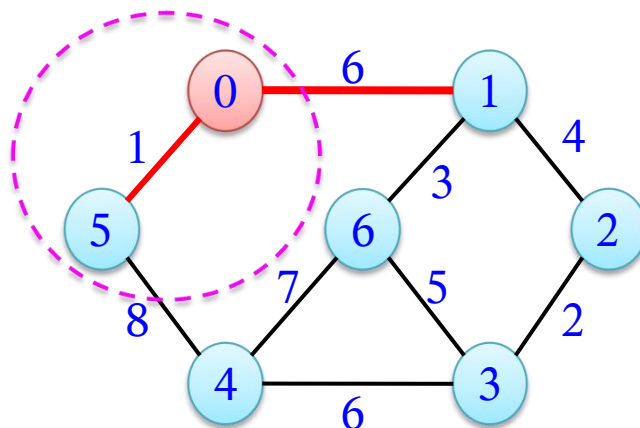
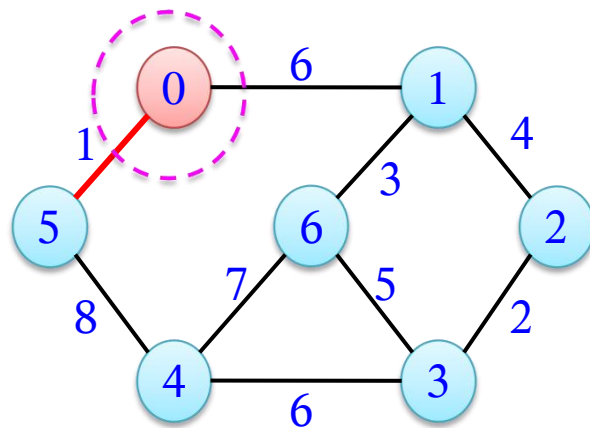
4.3 求解图问题

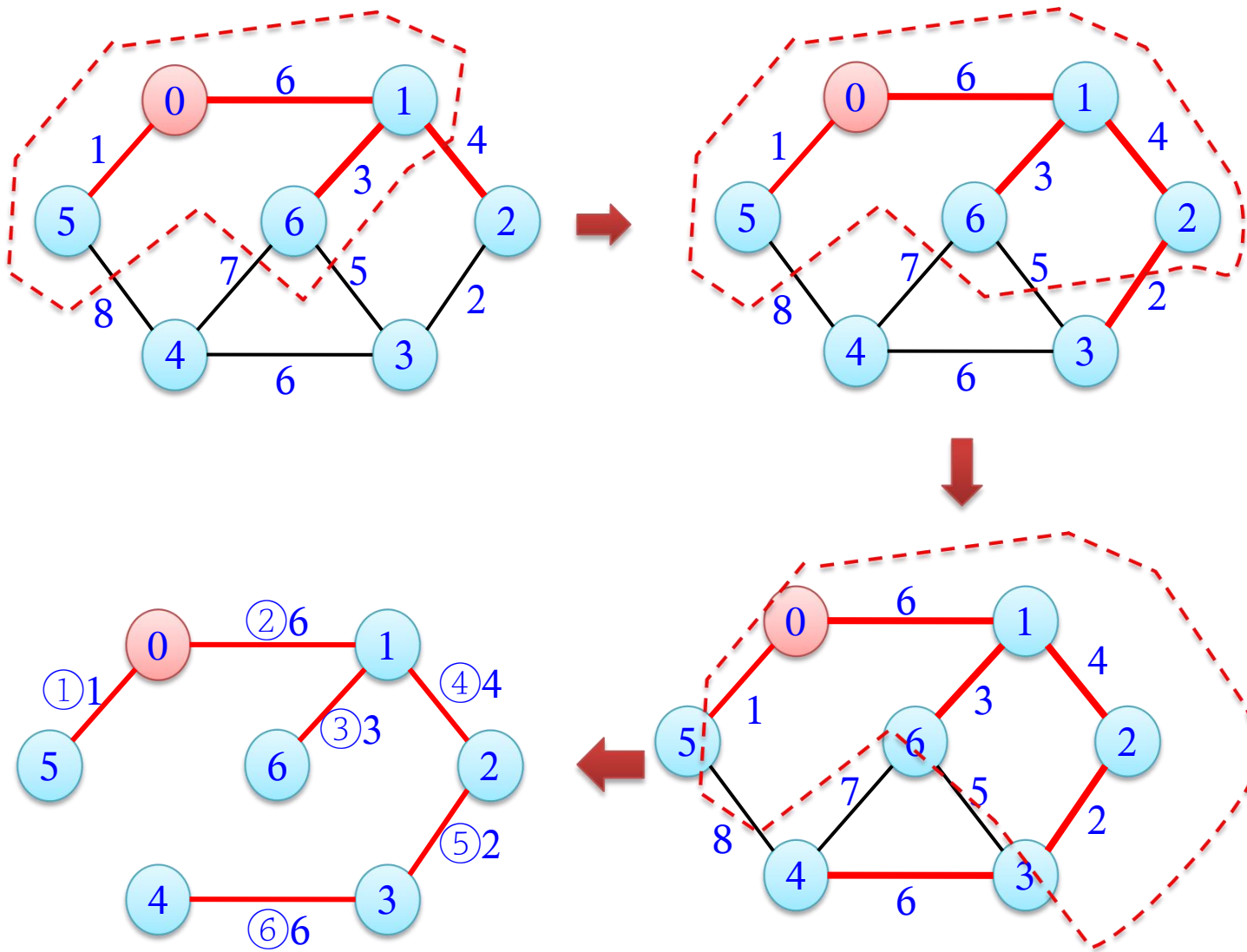
4.3.1 Prim算法构造最小生成树

- Prim (普里姆) 算法是一种构造性算法。
- 假设 $G = (V, E)$ 是一个具有 n 个顶点的带权连通图, $T = (U, TE)$ 是 G 的最小生成树, 其中 U 是 T 的顶点集, TE 是 T 的边集, 由 G 构造从起始顶点 v 出发的最小生成树 T 。

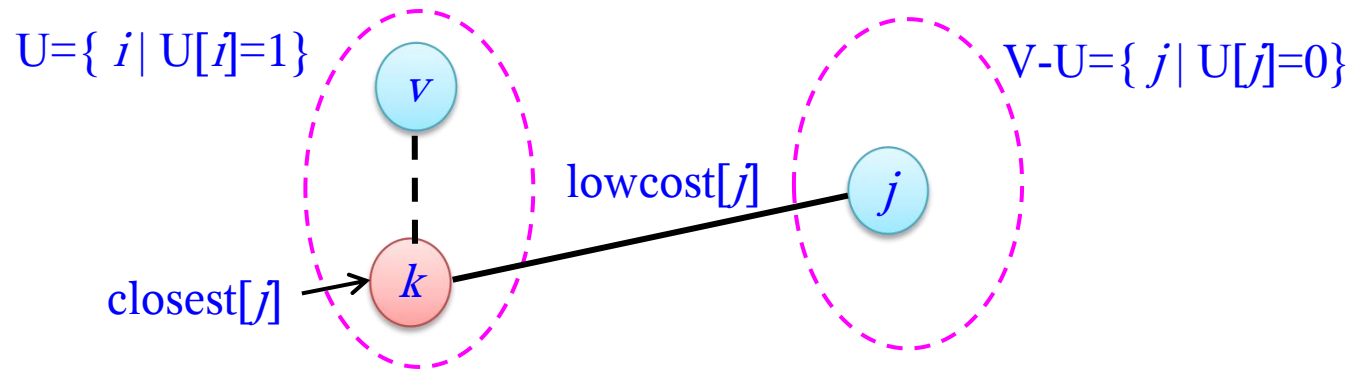


(a) 一个带权连通图





一棵最小生成树



```

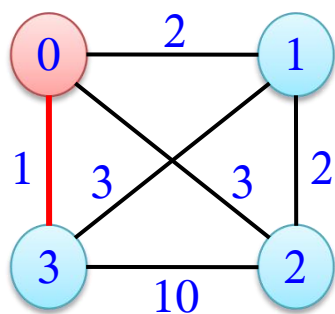
vector<vector<int>> Prim(vector<vector<int>>& A,int v)
{
    vector<vector<int>> T;           //T存放最小生成树，每条边为{起点,终点,边权值}
    int n=A.size();
    vector<int> lowcost(n,INF);
    vector<int> U(n,0);
    vector<int> closest(n);
    int mincost;
    for (int j=0;j<n;j++)           //初始化lowcost和closest数组
    {
        lowcost[j]=A[v][j];
        closest[j]=v;
    }
    U[v]=1;                         //源点v加入U
}

```

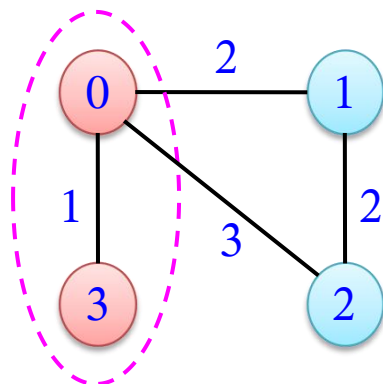
for (int i=1;i<n;i++)	//找出(n-1)个顶点
{ mincost=INF;	
int k=-1;	
for (int j=0;j<n;j++)	//在(V-U)中找出离U最近的顶点k
{ if (U[j]==0 && lowcost[j]<mincost)	
{ mincost=lowcost[j];	
k=j;	//k记录最近顶点的编号
}	
}	
T.push_back({closest[k],k,mincost});	//产生最小生成树的一条边
U[k]=1;	//顶点k加入U
for (int j=0;j<n;j++)	//修改数组lowcost和closest
{ if (U[j]==0 && A[k][j]<lowcost[j])	
{ lowcost[j]=A[k][j];	
closest[j]=k;	
}	
}	
}	
return T;	
}	

【算法分析】 Prim()算法中有两重for循环，所以时间复杂度为 $O(n^2)$

Prim算法是一种贪心算法，如何理解Prim算法的最优子结构性质呢？



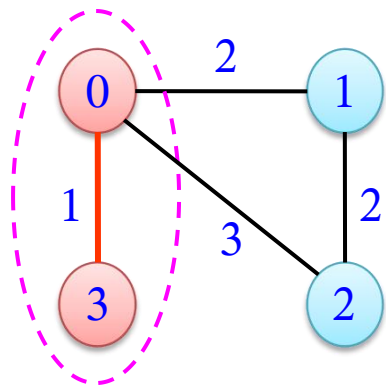
(a) 一个带权连通图



(b) 子问题

采用反证法证明Prim算法满足**最优子结构性质**。

证明：如果采用Prim算法求出原问题的生成树 T 是最小生成树，选择第一条边 (v, a) 后得到相应的子问题，假设该子问题采用Prim算法求出的生成树 T_1 ($T=(v, a) \cup T_1$) 不是最小的，而是最小生成树为 T_2 ，则 $(v, a) \cup T_2$ 得到原问题的一棵不同于 T 的更小的最小生成树，用 T 是原问题的最小生成树矛盾。最优子结构性质即证。



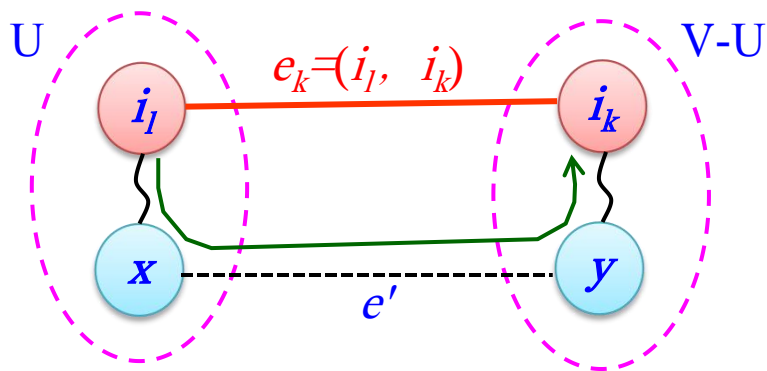
(b) 子问题

贪心选择性质证明。

命题4.1 对于任意正整数 $k < n$ ，存在一棵最小生成树 T 包含Prim算法前 k 步选择的边。

证明： $k=1$ 时，由前面最优子结构性质证明可以得出命题是成立的。

- 假设算法进行了 $k-1$ 步，生成树的边为 $e_1, e_2, \dots, e_{k-1}$ ，这些边的 k 个顶点构成集合 U ，并且存在 G 的一棵最小生成树 T 包含这些边。算法第 k 步选择了顶点 i_k ，则 i_k 到 U 中顶点的边的权值最小，设这条边为 $e_k=(i_l, i_k)$ 。
- 假设最小生成树 T 不含有边 e_k ，将 e_k 添加到 T 中形成一个回路，如下图所示，这个回路一定有连接 U 与 $V-U$ 中顶点的边 e' ，用 e_k 替换 e' 得到树 T' ，即 $T'=(T-\{e'\})\cup\{e_k\}$ 。



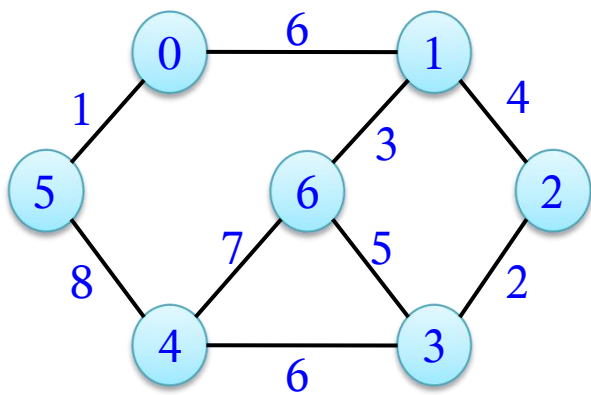
- 则 T' 也是一棵生成树，包含边 $e_1, e_2, \dots, e_{k-1}, e_k$ ，并且 T' 所有边权值和更小（除非 e' 与 e_k 的权相同），与 T 为一棵最小生成树矛盾，命题7.1即证。

命题4.1 对于任意正整数 $k < n$, 存在一棵最小生成树 T 包含Prim算法前 k 步选择的边。

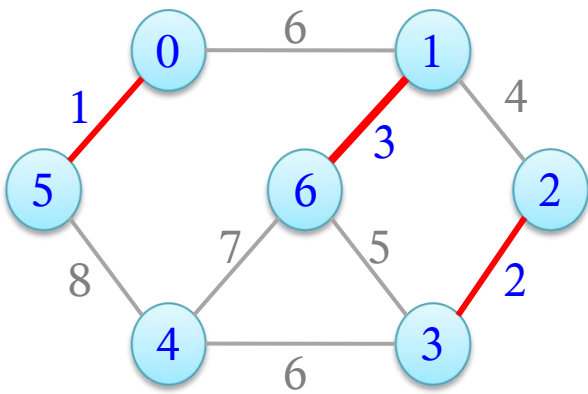
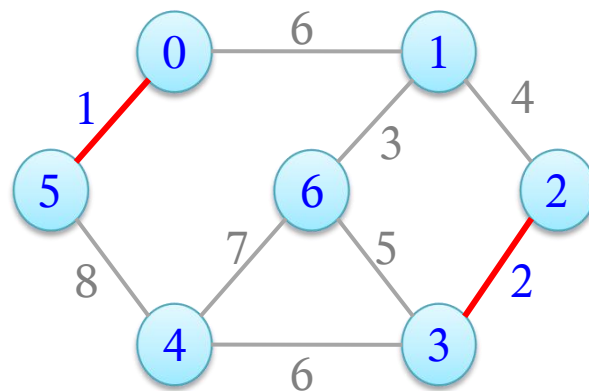
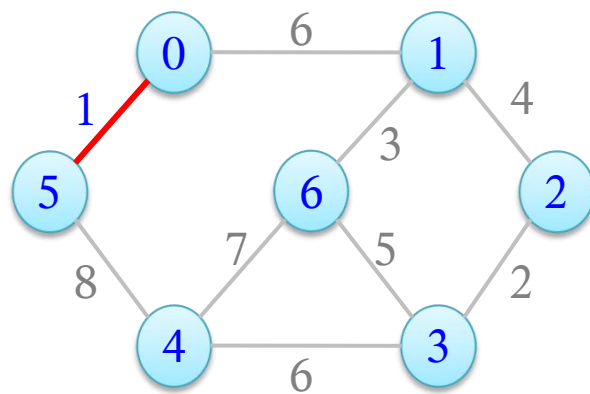
当命题7.1成立时, $k=n-1$ 即选择了 $n-1$ 条边, 此时 U 包含 G 中所有顶点, 由Prim算法构造的 $T=(U, TE)$ 就是 G 的最小生成树。

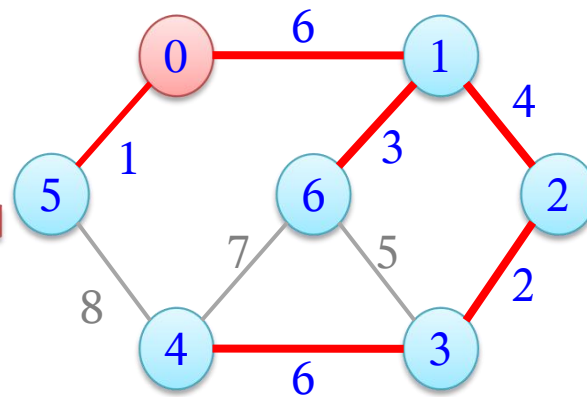
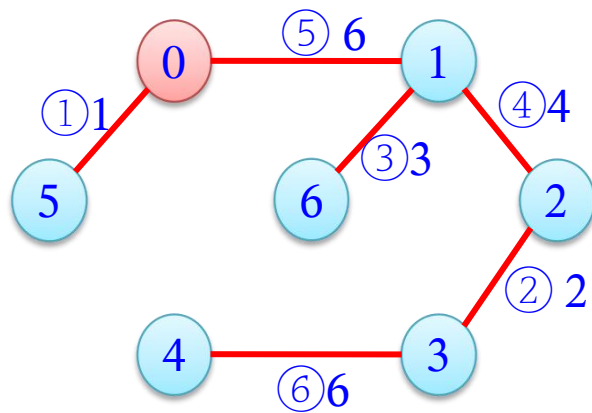
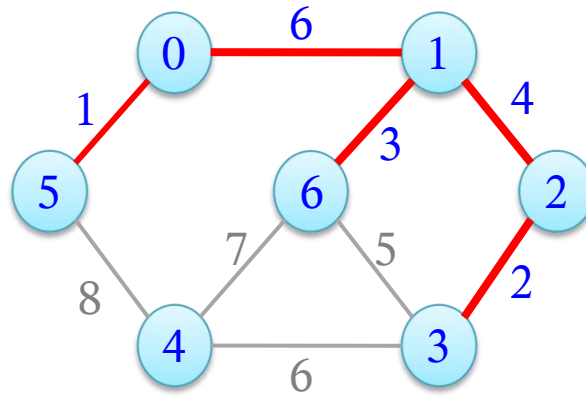
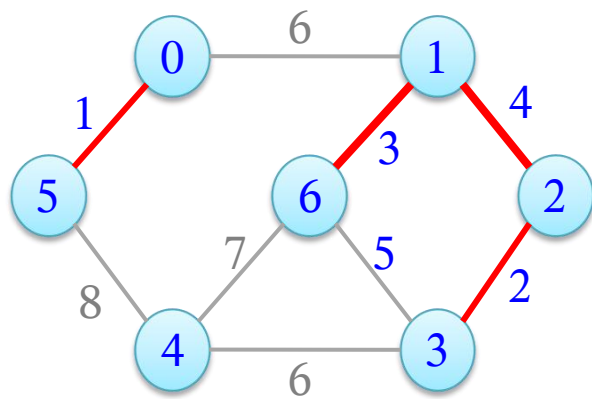
4.3.2 Kruskal算法构造最小生成树

- Kruskal（克鲁斯卡尔）算法按权值的递增次序选择合适的边来构造最小生成树。
- 假设 $G=(V, E)$ 是一个具有 n 个顶点 e 条边的带权连通无向图， $T=(U, TE)$ 是 G 的最小生成树。



(a) 一个带权连通图





一棵最小生成树

- 首先， U 中包含全部顶点， TE 为空，看成是由 n 个连通分量构成的图，每个连通分量中只有一个顶点，当考虑一条边 (u, v) 时，若 u 和 v 属于两个不同的连通分量，则加入该边不会出现回路，否则会出现回路。这里每个连通分量就是并查集中的子集树。
- 用数组 E 存放图 G 中的所有边，按权值递增排序，再从头到尾依次考虑每一条边，若可以加入则选择该边作为最小生成树的一条边，否则舍弃该边。

```
int parent[MAXN];
```

//并查集存储结构

```
int rnk[MAXN];
```

//存储结点的秩(近似于高度)

```
void Init(int n)
```

//并查集初始化

```
{ for (int i=0;i<n;i++)
```

```
{ parent[i]=i;
```

```
  rnk[i]=0;
```

```
}
```

```
}
```

```
int Find(int x)
```

//递归算法：并查集中查找x结点的根结点

```
{ if (x!=parent[x])
```

```
    parent[x]=Find(parent[x]);
```

//路径压缩

```
    return parent[x];
```

```
}
```

```
void Union(int x,int y)
```

```
{  int rx=Find(x);
```

```
    int ry=Find(y);
```

```
    if (rx==ry)
```

```
        return;
```

```
    if (rnk[rx]<rnk[ry])
```

```
        parent[rx]=ry;
```

```
    else
```

```
    {  if (rnk[rx]==rnk[ry])
```

```
        rnk[rx]++;
```

```
        parent[ry]=rx;
```

```
    }
```

```
}
```

//并查集中x和y的两个集合的合并

//x和y属于同一棵树的情况

//rx结点作为ry的孩子

//秩相同，合并后rx的秩增1

//ry结点作为rx的孩子

```
struct Edge
```

```
{   int u;
```

```
    int v;
```

```
    int w;
```

```
    Edge(int u,int v,int w):u(u),v(v),w(w) { }
```

```
    bool operator<(const Edge &b) const
```

```
    {
```

```
        return w<b.w;
```

```
    }
```

```
};
```

//边类型

//边的起点

//边的顶点

//边的权值

//用于按w递增排序

```

vector<vector<int>> Kruskal(vector<vector<int>>& A)  //Kruskal算法
{
    vector<vector<int>> T;           //T存放最小生成树，每条边为{起点,终点,边权值}
    int n=A.size();
    vector<Edge> E;
    for (int i=0;i<n;i++)           //由A下三角部分产生的边集E
    {
        for (int j=0;j<i;j++)
        {
            if (A[i][j]!=0 && A[i][j]!=INF)
                E.push_back(Edge(i,j,A[i][j]));
        }
    }
}

```

```

sort(E.begin(),E.end());
Init(n);
int k=0;
int j=0;
while (k<n-1)
{
    int u1=E[j].u;
    int v1=E[j].v;
    int sn1=Find(u1);
    int sn2=Find(v1);
    if (sn1!=sn2)
    {
        T.push_back({u1,v1,E[j].w});
        k++;
        Union(u1,v1);
    }
    j++;
}
return T;
}

```

```

//按w递增排序
//初始化并查集
//k表示生成树的边数
//E中边的下标,初值为0
//生成的边数小于n-1时循环

//取一条边的头尾顶点编号u1和v2

//分别得到两个顶点所属的集合编号
//添加该边不会构成回路
//产生最小生成树的一条边
//生成边数增1
//将u1和v1两棵子树合并

//遍历下一条边

```

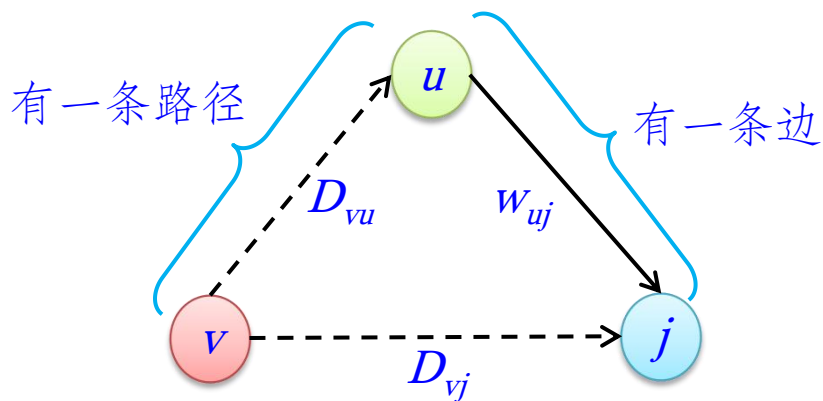
【算法分析】排序时间为 $O(e \log_2 e)$ ，while循环是在 e 条边中选取 $n-1$ 条边，最坏情况下执行 e 次，Union的执行时间为 $O(\log_2 n)$ ，所以上述Kruskal算法构造最小生成树的时间复杂度为 $O(e \log_2 e)$ 。

【Kruskal算法的正确性证明】和Prim算法一样都是贪心算法，其正确性证明与Prim算法类似，这里不再详述。

4.3.4 Dijkstra算法

- 设 $G=(V, E)$ 是一个带权有向图，所有边的权值为正数，给定一个源点 v ，求 v 到图中其他顶点的最短路径长度。
- Dijkstra（狄克斯特拉）算法把图中顶点集合 V 分成两组，第1组为已求出最短路径的顶点集合（用 S 表示），初始时 S 中只有一个源点 v ，第2组为其余未求出最短路径的顶点集合（用 U 表示）。
- 每求得一条最短路径 $v, \dots, u$ ，就将 u 加入到集合 S 中（重复 $n-1$ 次），直到全部顶点都加入到 S 中。

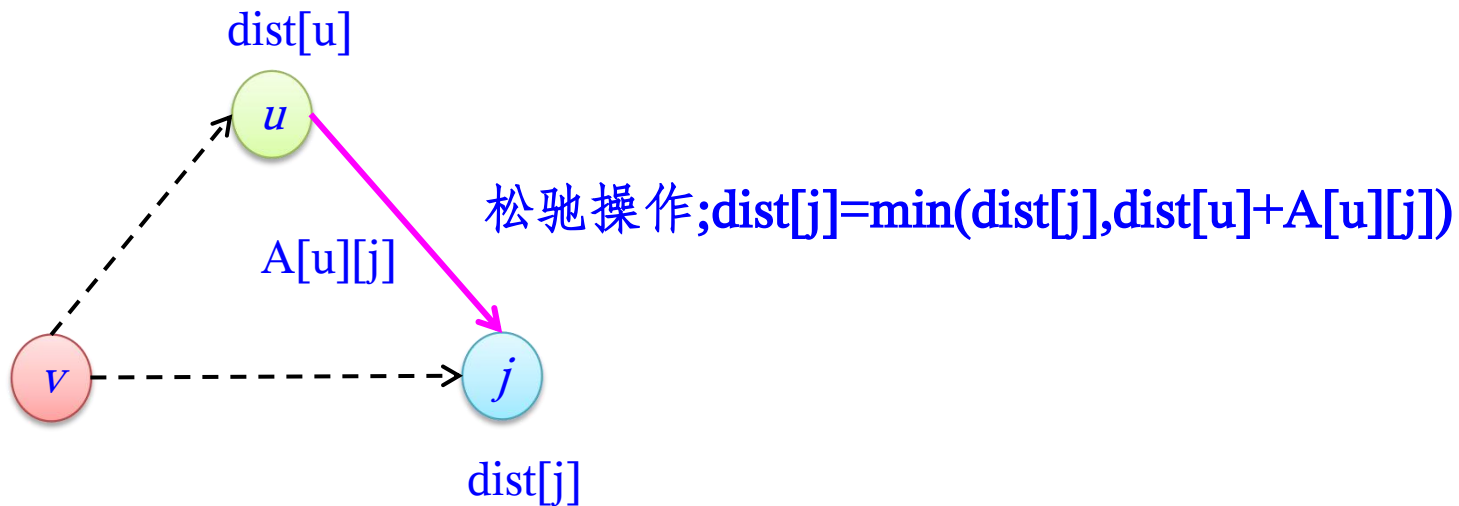
- 在向S中添加顶点 u 时，对于U中的每个顶点 j ，如果顶点 u 到顶点 j 有边（权值为 w_{uj} ），且原来从顶点 v 到顶点 j 的路径长度（ D_{vj} ）大于从顶点 v 到顶点 u 的路径长度（ D_{vu} ）与 w_{uj} 之和，即 $D_{vj} > D_{vu} + w_{uj}$ ，则将 $v \cdots u \rightarrow j$ 的路径作为 v 到 j 的新最短路径，即 $D_{vj} = \min(D_{vj}, D_{vu} + w_{uj})$ 。



$$D_{vj} = \min(D_{vj}, D_{vu} + w_{uj})$$

带权有向图用邻接矩阵 A 表示

$\text{dist}[i]$ 表示从源点 v 到顶点 i 的目前最短路径长度



```
vector<int> Dijkstra(vector<vector<int>>& A,int v)
```

//Dijkstra算法

```
{   int n=A.size();
```

```
    vector<int> dist(n); vector<bool> S(n,false);
```

```
    for (int i=0;i<n;i++) dist[i]=A[v][i];
```

```
    S[v]=true;
```

```
    for (int i=0;i<n-1;i++)
```

//距离初始化

//源点v放入S中

//循环n-1次

```
    {   int u=-1;
```

```
        int mindis=INF;
```

```
        for (int j=0;j<n;j++)
```

//选取U中最小点u

```
        {   if (S[j]==0 && dist[j]<mindis)
```

```
            {   u=j;
```

```
                mindis=dist[j];
```

```
            }
```

```
        }
```

```
        if(u==-1) break;
```

```
        S[u]=true;
```

//顶点u加入S中

```
        for (int j=0;j<n;j++)
```

//修改U中的顶点的距离

```
        {   if (!S[j] && A[u][j]!=0 && A[u][j]<INF)
```

```
            dist[j]=min(dist[j],dist[u]+A[u][j]);
```

```
        }
```

```
    }
```

```
    return dist;
```

【算法分析】 Dijkstra()算法中时间复杂度为 $O(n^2)$

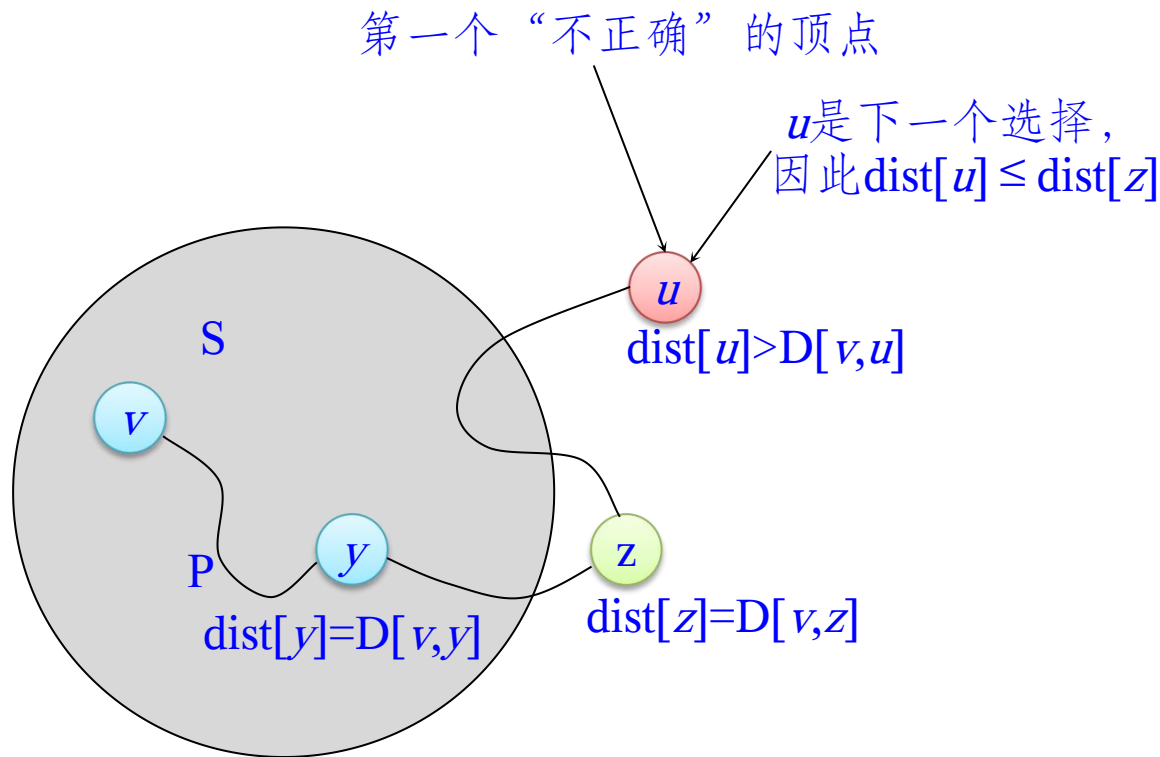
```
}
```

Dijkstra算法的正确性证明转换为证明以下命题成立。

命题4.2 Dijkstra算法中当顶点 u 添加的 S 中时, $\text{dist}[u]$ 等于从源点 v 到 u 的最短路径长度 $D[v, u]$ ($D[i, j]$ 表示图中从顶点 i 到 j 的真实最短路径长度)。

证明: 假设对于 V 中的某个顶点 t , $\text{dist}[t] > D[v, t]$, 并设 u 是算法中添加
到 S 中的第一个满足 $\text{dist}[u] > D[v, u]$ 的顶点。

因为存在从源点 v 到 u 的最短路径 P , 考虑当 u 添加到 S 的时刻, 令 z 是此时不在 S 中的 P 路径上的第一个顶点。令 y 是路径 P 中 z 的前一个顶点 (可能有 $y=v$), 如图所示。



现在选择的是将 u 添加的 S 中而不是 z ，因此有 $\text{dist}[u] \leq \text{dist}[z]$ （按照Dijkstra算法，越先添加到 S 中其 dist 越小）。

显然任何一条最短路径的子路径也是最短路径，因此，由于 z 在从 v 到 u 的最短路径上，则有 $D[v, z] + D[z, u] = D[v, u]$ 。此外， $D[z, u] \geq 0$ （因为图中没有负权），因此：

$$\text{dist}[u] \leq \text{dist}[z] = D[v, z] \leq D[v, z] + D[z, u] = D[v, u]$$

这样与 u 的定义矛盾。因此这样的顶点 u 不存在。命题7.2即证。

4.3.5 实战—最短路径问题 (HDU3790)

问题描述：有 n 个点， m 条无向边的图，每条边都有长度 d 和花费 p ，再给一个起点 s 和一个终点 t ，要求输出起点到终点的最短距离及其花费，如果最短距离有多条路线，则输出花费最少的。

输入描述：输入 n 、 m ，顶点的编号是 $1\sim n$ ，然后是 m 行，每行4个数 a 、 b 、 d 、 p ，表示顶点 a 和 b 之间有一条边，且其长度为 d ，花费为 p 。最后一行是两个数 s 、 t ，表示起点 s 和终点 t 。 n 和 m 为0时输入结束
($1 < n \leq 1000$, $0 < m < 100000$, $s \neq t$)。

输出描述：输出一行有两个数，分别表示最短距离及其花费。

解

利用 **Dijkstra** 求顶点 s 到顶点 t 的花费最小的最短路径，注意结果路径中首先是路径长度最短，而最短路径可能有多条，如果有多条取花费最小的一条路径。

这样对 **Dijkstra** 算法中做两点修改：

- 增加记录路径最小花费的数组 cost ， $\text{cost}[j]$ 表示从顶点 s 到顶点 j 的最短路径的最小花费，当存在多条最短路径时，需要比较路径花费求 $\text{cost}[j]$ 。
- 一旦顶点 t 的最短路径已求出，就不需要考虑其他顶点，直接输出结果并结束以提高时间性能。

```
#include<iostream>
#include<cstring>
using namespace std;
#define MAXV 1010
#define INF 0x3f3f3f3f //定义 ∞
int n,m;
int A[MAXV][MAXV],C[MAXV][MAXV];
int s,t;
```



```
void Dijkstra(int s)
```

```
//Dijkstra算法
```

```
{  int dist[MAXV];
```

```
    int cost[MAXV];
```

```
    int S[MAXV];
```

```
    int mindist,mincost,u;
```

```
    for(int i=1;i<=n;i++)
```

```
//dist、cost、S初始化,顶点编号从1开始
```

```
    {  dist[i]=A[s][i];
```

```
        cost[i]=C[s][i];
```

```
        S[i]=0;
```

```
    }
```

```
    dist[s]=cost[s]=0;
```

```
    S[s]=1;
```

```
    for(int i=0;i<n-1;i++)
```

```
    {  mindist=INF;
```

```
        for(int j=1;j<=n;j++)
```

```
//求V-S中的最小距离mindist
```

```
        {  if (S[j]==0 && mindist>dist[j])
```

```
            mindist=dist[j];
```

```
        }
```

```
    if (mindist==INF) break;
```

```
//找不到相邻顶点（提取结束）
```

```
    mincost=INF; u=-1;
```

```

for(int j=1;j<=n;j++)                                //求尚未考虑的、距离最小的顶点u
{
    if (S[j]==0 && mindist==dist[j] && mincost>cost[j])
    {
        mincost=cost[j];                            //在dist为最小的顶点中找最小cost的顶点u
        u=j;
    }
}
S[u]=1;                                                //将顶点u加入S集合
for (int j=1;j<=n;j++)                                //考虑顶点u,求s到顶点j的最短长度和花费
{
    int d=mindist+A[u][j];                            //d记录经过顶点u的路径长度
    int c=cost[u]+C[u][j];                            //c记录经过顶点u的花费
    if(S[j]==0 && d<dist[j])
    {
        dist[j]=d;
        cost[j]=c;
    }
    else if(S[j]==0 && d==dist[j] && c<cost[j])
        cost[j]=c;                                    //有多条长度相同的最短路径
}

```

```
if(S[t]==1)                                //已经求出s到t的最短路径
{   printf("%d %d\n",dist[t],cost[t]);
    return;
}
}
```

```

int main()
{
    int a,b,d,p;
    while(scanf("%d%d",&n,&m)!=EOF)
    {
        if(m==0 && n==0) break;
        memset(A,0x3f,sizeof(A));
        for(int i=0;i<m;i++)
        {
            scanf("%d%d%d%d\n",&a,&b,&d,&p);
            if(A[a][b]>d)                //可能有重复边，取长度较小者(否则出错)
            {
                A[a][b]=A[b][a]=d;      //无向图的边是对称的
                C[a][b]=C[b][a]=p;
            }
        }
        scanf("%d%d",&s,&t);
        Dijkstra(s);
    }
    return 0;
}

```

Realtime Status

From Problem ID Author Language Status

Run ID	Submit Time	Judge Status	Pro.ID	Exe.Time	Exe.Memory	Code Len.	Language	Author
36733284	2021-09-12 16:05:43	Accepted	3790	249MS	9696K	2043 B	C++	licb1964

4.4 求解调度问题

- 调度问题有许多形式，这里专指这样形式的调度问题， n 个作业要在-台机器上加工，每个作业的加工时间可能不同，这样有些作业就需要等待，全部作业完工的时间为等待时间和加工时间之和，称为系统总时间。
- 该调度问题通常有两种，一是不带惩罚，另外一种是带惩罚的。

4.4.1 不带惩罚的调度问题

- 不带惩罚的调度问题的最优解是最小系统总时间，实际上 n 个作业的加工顺序不同对应的系统总时间也不相同，该问题就是求一个具有最小系统总时间的加工顺序。
- **贪心策略**是选择当前加工时间最小的作业优先加工，也就是按加工时间递增排序，再按排序后的顺序依次加工。

序号 i	作业编号 no	加工时间 t_i	等待时间 w_i	总时间 s_i
0	0	5	0	5
1	1	3	5	8
2	2	4	8	12
3	3	2	12	14



按加工时间递增排序

序号 i	作业编号 no	加工时间 t_i	等待时间 w_i	总时间 s_i
0	3	2	0	2
1	1	3	2	5
2	2	4	5	9
3	0	5	9	14



系统总时间 $T=2+5+9+14=30$

```
int greedily(vector<int>&a)
{
    sort(a.begin(),a.end());
    int T=0;
    int w=0;
    int n=a.size();
    for(int i=0;i<n;i++)
    {
        T+=a[i]+w;
        w+=a[i];
    }
    return T;
}
```

//当前系统总时间
//当前作业的等待时间
//n个作业

【算法分析】 算法的执行时间主要花费在排序上，对应的时间复杂度为 $O(n\log_2 n)$ 。

正确性证明：略。

4.4.2 带惩罚的调度问题

- 带惩罚的调度问题中，通常假设 n 个作业加工时间均为一个时间单位，时间用 $0 \sim \text{maxd}$ 的连续整数表示，每个作业有一个截止时间（deadline用时间整数表示），当一个作业在其截止时间之后完成，对应有一个惩罚值（punish）。
- 该问题的最优解是最小总惩罚值。

贪心策略：选择当前惩罚值最大的作业优先加工，按惩罚值递减排序，并且尽可能选择作业截止时间之前最晚的时间加工。按排序后的顺序依次加工。

作业编号no	截止时间 d_i	惩罚值 p_i
0	4	70
1	2	60
2	4	50
3	3	40
4	1	30
5	4	20
6	6	10

$days[i]$ 表示时间 i 是否在加工,初始均为F

选时间4, $days[4]=T$, 不惩罚, $ans=0$

选时间2, $days[2]=T$, 不惩罚, $ans=0$

选时间3, $days[3]=T$, 不惩罚, $ans=0$

选时间1, $days[1]=T$, 不惩罚, $ans=0$

时间1已占, 不能加工, 惩罚, $ans=30$

时间1~4已占, 不能加工, 惩罚,
 $ans=30+20=50$

选时间6, $days[6]=T$, 不惩罚, $ans=50$

```
struct Job                                //作业类型
{
    int dtime;                            //截止时间
    int punish;                           //惩罚值
    bool operator<(const Job& b) const
    {
        return punish>b.punish;          //按punish递减排序
    }
};
```

```

int greedily(vector<Job>&a)
{
    int n=a.size();
    int maxd=0;
    for(int i=0;i<n;i++)
        maxd=max(maxd,a[i].dtime)
    vector<bool> days(maxd,false);
    sort(a,a+n);
    int ans=0;
    for(int i=0;i<n;i++)
    {
        int j=a[i].dtime;
        for(;j>0;j--)
        {
            if(days[j]==false)
            {
                days[j]=true;
                break;
            }
        }
        if(j==0)
            ans+=a[i].punish;
    }
    return ans;
}

```

//求最大的截止时间

//排序

//查找截止日期之前的空时间
//找到空时间

//没有找到空时间
//累计惩罚值

7.4.3 实战—赶作业 (HDU1789)

问题描述：A有 n 份作业要做，每份作业有一个最后期限，如果在最后期限后交作业老师就会扣分，现在假设完成每份作业都需要一天。所以A想到了要安排做作业的顺序，把扣分降到最低，请你帮助他。

输入格式：输入包含 t 个测试用例，第一行是单个整数 t 。每个测试用例以一个正整数 n 开头 ($1 \leq n \leq 1000$)，表示作业的数量，然后两行，第一行包含 n 个整数，表示作业的截止日期，下一行包含 n 个整数，表示作业的扣分。

输出格式：对于每个测试用例，在一行中输出最少的扣分。

解

属于典型的带惩罚的调度问题，实际上扣分就是惩罚值，最优解就是求最小的总惩罚值。

```

#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
#define MAXN 1010

struct Job
{
    int dtime;
    int punish;
    bool operator<(const Job& b) const
    {
        return punish>b.punish;
    }
};
int n;
Job a[MAXN];

```

//作业的类型
//截止日期
//扣分
//按punish递减排序

int greedly (int maxd)	//贪心算法
{ vector<bool> days(maxd,false);	
sort (a,a+n);	//排序
int ans =0;	
for(int i=0;i<n;i++)	
{ int j=a[i].dtime;	
for(;j>0;j--)	//查找截止日期之前（含）的空时间
{ if(days [j]==false)	//找到空时间
{ days[j]=true;	
break;	
}	
}	
if(j==0)	//没有找到空时间
ans +=a[i].punish;	//累计扣分
}	
return ans ;	
}	


```

int main()
{
    int t;
    scanf("%d",&t);
    while(t--)
    {
        int maxd=0;
        scanf("%d",&n);
        for(int i=0;i<n;i++)
        {
            scanf("%d",&a[i].dtime);
            maxd=max(maxd,a[i].dtime);
        }
        for(int i=0;i<n;i++)
            scanf("%d",&a[i].punish);
        printf("%d\n",greedily(maxd));
    }
    return 0;
}

```

- 上述程序提交时通过，执行时间为46ms，内存消耗为1748KB。
- `greedly`算法的时间复杂度为 $O(n * \max d)$ ，可以采用并查集改进时间性能。
- 设置并查集数组`parent`，`parent[i]`表示第 i 天的前一个空时间（含第 i 天），初始时置`parent[i]=i`，当第 i 天被占用时合并操作是置`parent[i]=i-1`（可以理解为第 i 天被占用，前面的空时间与第 $i-1$ 天前面的空时间相同），由于这里的合并有方向性，所以不用`rnk`数组。

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
#define MAXN 1010
vector<int> parent;
vector<int> rnk;

int Find(vector<int>& parent,int x)
{   if (x!=parent[x])
        parent[x]=Find(parent,parent[x]);
    return parent[x];
}
```

//并查集存储结构

//存储结点的秩(近似于高度)

//递归算法：查找x结点的根结点

//路径压缩

```
struct Job
{
    int dtime;
    int punish;
    bool operator<(const Job& b) const
    {
        return punish>b.punish;
    }
};
int n;
Job a[MAXN];
```

//作业的类型
//截止日期
//扣分

//按punish递减排序

int greedly (int maxd)	//贪心算法
{ parent.resize(maxd);	
for(int i=0;i<=maxd;i++)	//查集初始化
parent[i]=i;	
sort (a,a+n);	//排序
int ans =0;	
for(int i=0;i<n;i++)	
{ int day = Find (parent,a[i].dtime);	//查找截止日期之前的空时间
if(day>0)	//找到空时间
parent[day]=day-1;	//合并
else	
ans +=a[i].punish;	//累计扣分
}	
return ans ;	
}	

```

int main()
{
    int t;
    scanf("%d",&t);
    while(t--)
    {
        int maxd=0;
        scanf("%d",&n);
        for(int i=0;i<n;i++)
        {
            scanf("%d",&a[i].dtime);
            maxd=max(maxd,a[i].dtime);
        }
        for(int i=0;i<n;i++)
            scanf("%d",&a[i].punish);
        printf("%d\n",greedily(maxd));
    }
    return 0;
}

```

上述程序提交时通过，执行时间为31ms，内存消耗为1748KB。

↑
46ms

4.5 哈夫曼编码

4.5.1 哈夫曼树和哈夫曼编码

设需要编码的字符集为 $\{d_0, d_1, \dots, d_{n-1}\}$ ，它们出现的频率为 $\{w_0, w_1, \dots, w_{n-1}\}$ ，应用哈夫曼树构造最优的不等长的由0、1构成的编码方案（哈夫曼编码）。

构造一棵哈夫曼树

- ① 由给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构造 n 棵只有一个叶子结点的二叉树，从而得到一个二叉树的集合 $F=\{T_1, T_2, \dots, T_n\}$ 。
- ② 在 F 中选取根结点的权值最小和次小的两棵二叉树作为左、右子树构造一棵新的二叉树，这棵新的二叉树根结点的权值为其左、右子树根结点权值之和。即合并两棵二叉树为一棵二叉树。
- ③ 重复步骤②，当 F 中只剩下一棵二叉树时，这棵二叉树便是所要建立的哈夫曼树。

构造哈夫曼编码

- 利用哈夫曼树构造的用于通信的二进制编码称为哈夫曼编码。
- 哈夫曼树中从根到每个叶子都有一条路径，对路径上的各分支约定指向左子树根的分支表示“0”码，指向右子树的分支表示“1”码，取每条路径上的“0”或“1”的序列作为和各个叶子对应的字符的编码，这就是哈夫曼编码。

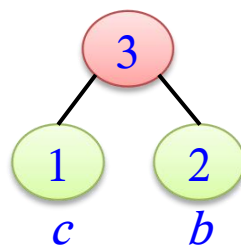
证明算法的正确性，也就是证明如下两个命题是成立的。

命题7.4 两个最小权值字符对应的结点 x 和 y 必须是哈夫曼树中最深的两个结点且它们互为兄弟。

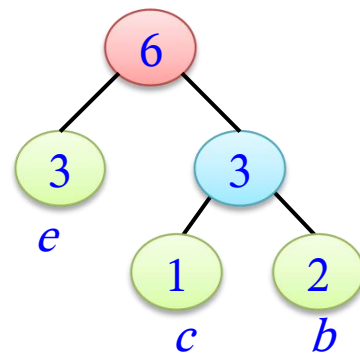
命题7.5 设 T 是字符集 C 对应的一棵哈夫曼树，结点 x 和 y 是兄弟，它们的双亲为 z ，显然有 $w_z = w_x + w_y$ ，现删除结点 x 和 y ，让 z 变为叶子结点，那么这棵新树 T_1 一定是字符集 $C_1 = C - \{x, y\} \cup \{z\}$ 的最优树。



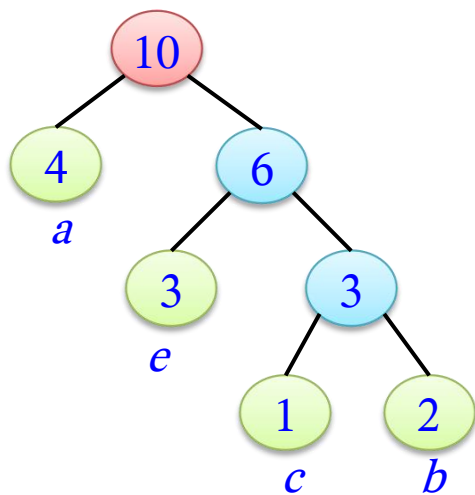
(a) 初始



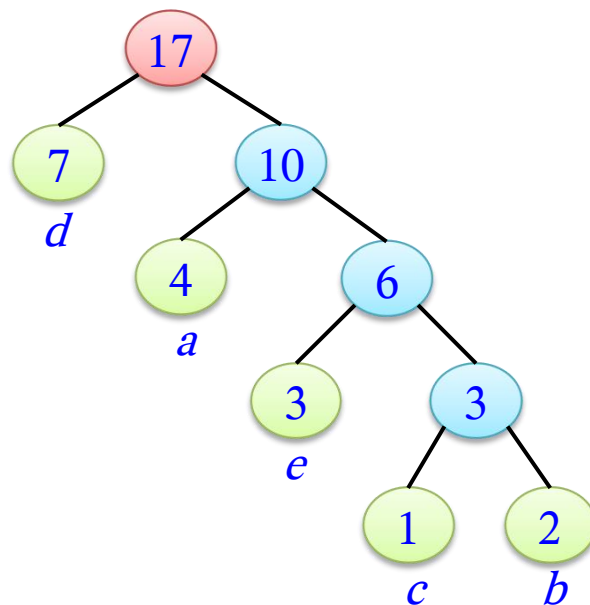
(b) 第1次合并



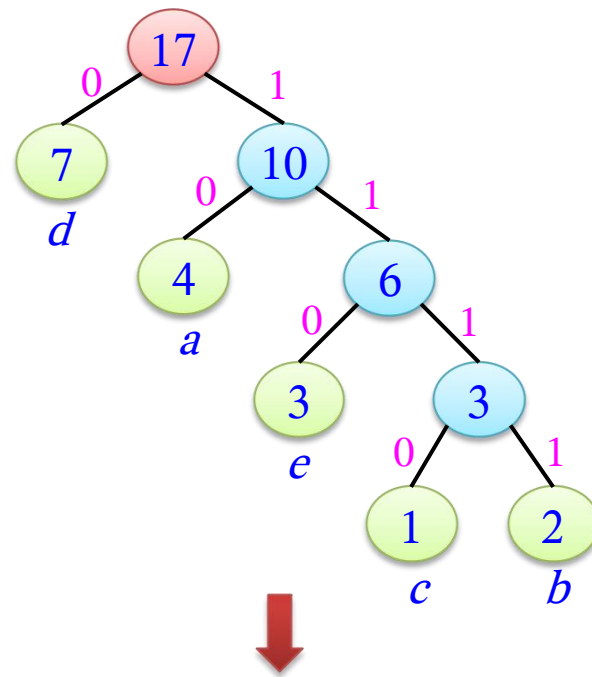
(c) 第2次合并



(d) 第3次合并



(e) 第4次合并



a: 10

b: 1111

c: 1110

d: 0

e: 110

$$\text{WPL} = (1+2) \times 4 + 3 \times 3 + 4 \times 2 + 7 \times 1 = 36$$

哈夫曼编码

4.6 最优装载问题

■ 问题描述

- 有 n 个集装箱要装上一艘**载重量为 c** 的轮船，其中集装箱 i 的重量为 w_i
- 要求：在**装载体积不受限制**的情况下，将**尽可能多的**集装箱装上轮船，且装载的集装箱总重量不超过 c
 - 优化目标：装载的集装箱数目最大化
 - 约束条件：装载重量受限

■ 形式化描述： $\max \sum_{i=1}^n x_i$

$$\sum_{i=1}^n w_i x_i \leq c$$

$$x_i \in \{0, 1\}, \quad 1 \leq i \leq n$$

$x_i = 0$ 表示不装入集装箱， $x_i = 1$ 表示装入集装箱

最优装载问题是0-1背包问题的1个特例：

集装箱=物品

每种物品价值函数
 $v[i]=1$

■ 算法描述

- 贪心选择策略：先装重量最轻者，船的剩余可装载（重量）容量最大化，以容纳更多的其它集装箱，产生最优解

解向量 各集装箱重量 总装载量 集装箱总数

```
void Loading(int x[], Type w[], Type c, int n)
{
    int *t = new int [n+1];
    Sort(w, t, n);
    for (int i = 1; i <= n; i++) x[i] = 0;    //解向量赋初值

    for (int i = 1; i <= n && w[t[i]] <= c; i++)
    {
        x[t[i]] = 1;
        c -= w[ t[i] ];
    }
}
```

//对n个集装箱，按照重量w[i]从小到大重新排序并排列编号，结果放在t中

// 在有剩余容量的前提下，从轻到重，装入各个集装箱

最优装载问题的贪心选择性质

最优装载问题具有贪心选择性质，证明如下。

- 设集装箱已按重量**从小到大**排序， $X=\{x_1, x_2, \dots, x_n\}$ 是最优装载问题的一个最优解

- 设最优解 X 中， k 为最先装入箱中的最轻货物，即

$$k = \min_{1 \leq i \leq n} \{i \mid x_i = 1\}$$

- E.g. 最优解为 $\{1, 1, 0, 1, 1\}$ ，则 $k=1$

最优解为 $\{0, 1, 1, 1, 1\}$ ，则 $k=2$

- 当问题有最优解（有可能是非贪心策略解）时，分两种情况
- 情况1：当 $k=1$ ，即最轻的第1个货物被装入，货物选择的放入顺序符合贪心策略， $\{x_1, x_2, \dots, x_n\}$ 是满足贪心选择性质的一个最优解

贪心选择性质

- 情况2：当 $k > 1$ 时，即最轻的集装箱1没有被装入

e.g. 最优解为 $\{0, 1, 1, 1, 1\}$ ， $k=2$

- 按如下方式，通过替换，从 $\{x_1, x_2, \dots, x_n\}$ 构造出一个满足贪心策略的最优解 $Y = \{y_1, y_2, \dots, y_k, \dots, y_n\}$ ， $y_1 = 1$

- i) 对 k ，其中 $1 < k < n$ ， $y_k = 1$ ，取 $y_1 = 1$ ， $y_k = 0$ ，将第1个集装箱放入，第 k 个集装箱拿出，即用第1个集装箱替换原方案中最轻的集装箱，而第1个集装箱比第 k 个集装箱更轻，即 $w_1 \leq w_k$
- ii) $y_i = x_i$ ， $1 \leq i \leq n$ ， $i \neq k$ ，其它集装箱是否被放入仍保持不变

e.g. 原最优解 $\{x_1, x_2, \dots, x_n\} = \{0, 1, 1, 1, 1\}$ ， $k=2$ ，改为：

$$\{y_1, y_2, \dots, y_n\} = \{1, 0, 1, 1, 1\}, \quad k=1$$

贪心选择性质

- 新方案仍然满足容量约束条件：

$$\sum_{i=1}^n w_i y_i = w_1 - w_k + \sum_{i=1}^n w_i x_i \leq \sum_{i=1}^n w_i x_i \leq c$$

- 因此，新方案Y是一个满足贪心策略的可行解。

- 需进一步判断装入的集装箱数是否最多？

- 由于 $\sum_{i=1}^n y_i = \sum_{i=1}^n x_i$

，说明第2个方案Y={y₁, y₂, ..., y_k, ..., y_n}具有与最优方案X一样的装箱数

- 因此，优先将最轻的第1个集装箱装入可以使得装载的总箱数最大化！Y是一个满足贪心策略的最优解

最优装载问题的最优子结构性质

最优装载问题具有最优子结构性质，证明如下

- 假设集装箱 $\{1, 2, \dots, n\}$ 已按重量从小到大排序
- 1. 原问题：待装集装箱为 $\{1, 2, \dots, n\}$ 、容量为 c ，
 $\{x_1, x_2, \dots, x_n\}$ 是其满足贪心策略的1个最优解，且 $x_1=1$
 - e.g. $\{x_1, x_2, \dots, x_n\} = \{1, 0, 1, 1, 1\}$, $k=1$
- 2. 则对子问题：待装集装箱为 $\{2, \dots, n\}$ 、容量为 $c-w_1$ ，
 $\{x_2, \dots, x_n\}$ 是一个最优解，
拿掉第1个集装箱，
从容量 c 中减去其重量 w_1
 - e.g. $\{x_2, \dots, x_n\} = \{0, 1, 1, 1\}$

最优子结构性质

原问题: n 个集装箱 $\{1, 2, 3, \dots, n\}$
容量 c

解: $(x_1, x_2, \dots, x_n)$
(最优)

子问题1: $\{1\}$
容量 c

解: $(x_1)=1$

子问题2: $n-1$ 个集装箱 $\{2, 3, \dots, n\}$
容量 $c-w_1$

解: $(x_2, \dots, x_n)$
(最优)

最优装载问题的贪心算法Loading

- 算法参见教科书
- 由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法Loading的正确性
- 算法复杂性
 - ▣ 算法Loading的主要计算量在于将集装箱依其重量从小到大排序，故算法的时间复杂性为 $O(n\log n)$